MINI PROJECT-1 · PROBLEM P6

# Entropy-Based Analysis of Encrypted Data

Project report · Group G6


| | |
|---|---|
| **Anupama** | Mathematical modelling |
| **Hari** | Implementation and coding |
| **Karthika** | Algorithm design |
| **Nisha** | Presentation and technical communication |
| **Rohit** | Project coordination and data experimentation |
| **Sreerag** | Documentation |
| **Vandana** | Testing and validation |


We built Entropy Lab, a menu-driven Python application that measures the Shannon entropy of data byte by byte and tests how far that statistic can tell plain, encoded and encrypted data apart, without decrypting anything.

Contents


1. Technical problem description
2. Mathematical model
3. Proposed solution
4. Solution flow diagram
5. Core technical components
6. Results
7. Demo screenshots
8. Individual contribution summary
9. GitHub repository and conclusion



github.com/harishankarsa454-code/G6-Entropy-Analysis · October 2026

# 1. Technical problem description

**Scenario.** Sherlock, a cybersecurity data analyst, holds a collection of data samples. Some are plain, some are transformed and some are encrypted. He must find out how each kind differs using only the statistics of the data itself, because decrypting it is not an option.

**Objective.** Build a mathematical measure of the uncertainty or randomness in a sample. Use it to compare samples, test whether it can tell kinds of data apart, and state clearly what it can and cannot conclude.

**Inputs and outputs.**

| Inputs | Outputs |
|---|---|
| Typed text, a PDF, any file, or one of 16 built-in samples | Frequency of each of the 256 byte values |
| Caesar shift, Vigenère key and a passphrase | Shannon entropy in bits per byte, and the change ΔH after each transformation |
| Choice of preprocessing: every byte, or letters A–Z only | Extra measures: min-entropy, order check, chi-square, index of coincidence |
| Window size, noise level and distribution settings for the experiments | Entropy map, sample-size and distribution results, and a verdict with reasons |

**Constraints and assumptions.**

- The work is limited to the statistical distribution of the data. No cipher is broken or decrypted.
- Python only, menu-driven, with no cryptography or other library that performs the core work. Counting, entropy and every transformation are written by hand.
- Each byte is treated as one symbol, so entropy is measured in bits per byte on a 0 to 8 scale.
- Keys are derived from a passphrase so every run repeats. Real systems would use random keys.

**Mathematical concepts.** Discrete probability distributions; Shannon entropy and its maximum, $\log_2$ of the number of symbols; min-entropy; the chi-square goodness-of-fit test; the index of coincidence; modular arithmetic for the substitution ciphers; and the effect of sample size on an estimated distribution.

**What the system is expected to achieve.** Measure any sample in the same unit, show how each transformation changes the distribution, decide how distinguishable the kinds of data are, and demonstrate where a high or low entropy reading misleads.

# 2. Mathematical model

Every sample is a sequence of N bytes, and each byte value x from 0 to 255 is one symbol. The program turns counts into a probability distribution and measures it.

**Probability of each byte value**, where f(x) is how many times x appears:

$$p(x) = \frac{f(x)}{N}$$

**Shannon entropy**, the average unpredictability per byte, in bits:

$$H(X) = -\sum_{x=0}^{255} p(x)\,\log_2 p(x), \qquad 0 \le H(X) \le \log_2 256 = 8$$

H is 0 when one value fills the sample and 8 when all 256 values are equally likely. With k distinct symbols the ceiling is $\log_2 k$: 4.70 for 26 letters and 6.02 for Base64's 65 symbols. A sample of N bytes can never exceed $\log_2 N$, which is why small samples read low.

**Change in entropy** after a transformation:

$$\Delta H = H(\text{output}) - H(\text{original})$$

**Worked example: HELLO.** N = 5, with H, E and O appearing once and L twice.

| Symbol | f(x) | p(x) | $-p \log_2 p$ |
|---|---|---|---|
| H | 1 | 0.20 | 0.4644 |
| E | 1 | 0.20 | 0.4644 |
| L | 2 | 0.40 | 0.5288 |
| O | 1 | 0.20 | 0.4644 |
| Total | 5 | 1.00 | 1.9219 |

A Caesar shift of 3 turns HELLO into KHOOR. The counts are the same (K, H and R once, O twice), so H stays 1.9219. Substitution renames symbols but keeps the distribution.

**Measures beyond Shannon entropy.**

$$H_{\min} = -\log_2 \max_x p(x)$$

Min-entropy is the worst case: how good the single best guess is. It is never larger than H.

$$\chi^2 = \sum_{x=0}^{255} \frac{(f(x) - E)^2}{E}, \qquad E = \frac{N}{256}$$

Chi-square measures how far the counts are from perfectly even. It is converted to a z-score with the Wilson–Hilferty approximation; random data lands between −3 and +3.

$$IC = \frac{\sum_{i=1}^{26} f_i(f_i - 1)}{L(L-1)}$$

The index of coincidence, over the L letters only, is the chance that two letters picked at random match: about 0.067 for English and 0.038 for random letters.

The **order check** is the entropy of the differences d(i) = (x(i+1) − x(i)) mod 256. Shannon entropy ignores order; this measure does not, so it falls sharply for sorted data or a counter.

# 3. Proposed solution

We treat every sample as a probability distribution over 256 byte values, put the same input through eight transformations, and measure each result with the same hand-written entropy function. Because one number can mislead, four further measures and an entropy map back it up, and every reading is compared with random data of the same size.

**How the mathematical model is incorporated.** Each formula in section 2 is a function in `entropy_core.py`. The frequency count builds f(x), the entropy function turns it into p(x) and sums $-p \log_2 p$, and the experiments call the same functions on modified data. Nothing is computed by a library.

**Major processing steps.**

1. Load the input and convert it to bytes. Optionally keep letters A–Z only.
2. Count each of the 256 byte values with a plain loop.
3. Turn counts into probabilities and compute H, min-entropy, the order check, chi-square and the index of coincidence.
4. Run the same bytes through Caesar, Vigenère, Rail fence, repeating-key XOR, one-time pad, RC4, Base64 and a random reference.
5. Measure every output the same way and compute ΔH.
6. Compare each reading with random data of the same size, and measure entropy window by window to build an entropy map.
7. Combine the measures into a verdict with reasons, and print every result with a plain-English explanation.

**How the output is produced.** Each menu option prints its tables, a labelled 0–8 gauge, text bar charts or a 16 × 16 byte map, then a "What this means" box. An optional web dashboard shows the same results visually.

**Assumptions.** One byte is one symbol. Samples under about 64 bytes are too small to judge. The thresholds used by the verdict (2.0, 5.5 and 6.5 bits, and 6.3 bits for the entropy map) were chosen by us from measured samples.

## Advantages

1. **Statistics only.** Any file is analysed byte by byte without decrypting it, exactly as the problem requires, so it works on data we cannot read.
2. **Fair comparison.** Every reading is compared with random data of the same size, so a small encrypted sample reading 7.2 bits is not mistaken for weak encryption.
3. **More than one number.** The order check and chi-square catch cases that fool Shannon entropy: sorted ciphertext keeps 7.85 bits of entropy but its order check falls to 0.73.
4. **Locates encrypted data.** The entropy map finds where a high-entropy block sits inside a larger file, not just one average for the whole file.
5. **Transparent and repeatable.** Every formula is hand-written and every key is derived from a passphrase, so each number can be traced and reproduced in the viva.

## Limitations

1. **Cannot identify encryption.** Encrypted, compressed and random data all read close to 8 bits, and no measure here can tell them apart.
2. **Fixed thresholds.** The verdict bands and the 6.3-bit map cut-off were set by us from samples, not derived from theory, so borderline data can be misclassified.
3. **Sample size.** Below about 1,000 bytes readings are unreliable, and the chi-square test needs at least 1,280 bytes for an expected count of 5 per value.
4. **Teaching ciphers.** Keys are derived from a passphrase and the one-time pad uses a seeded generator, so the results show statistics only and say nothing about real key strength.
5. **Byte-level view.** Each byte is one symbol, so patterns that span several bytes, such as repeated 16-byte blocks, are only partly visible.

# 4. Solution flow diagram

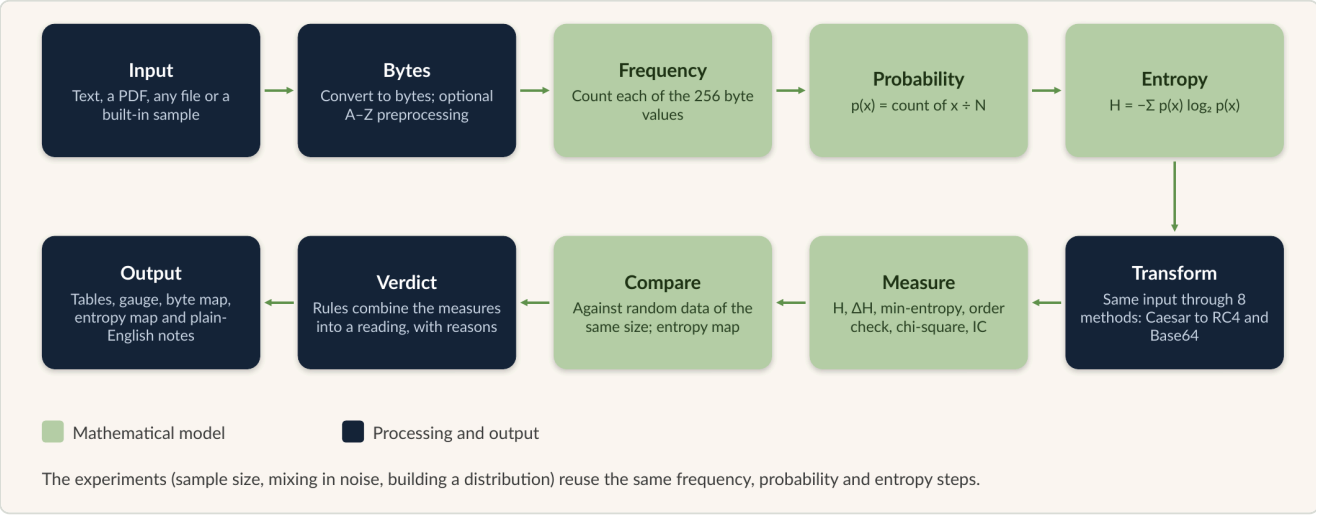



Figure 1: The solution flow. Green stages are the mathematical model; navy stages are processing and output. Every transformed output goes back through the same frequency, probability and entropy steps before it is compared.

# 5. Core technical components

| Component | Brief description | Technical significance | Mathematical significance |
|---|---|---|---|
| `count_bytes()` | Counts how often each of the 256 byte values appears, with a plain loop | Builds the frequency table every other measure starts from | Frequency f(x) of a discrete random variable |
| `entropy_from_counts()` | Turns counts into probabilities and sums $-p \log_2 p$ | The one measurement applied to every sample and output | Shannon entropy H(X), bounded by $\log_2 256 = 8$ |
| Transformations: `caesar`, `vigenere`, `rail_fence`, `xor_repeating`, `one_time_pad`, `rc4`, `base64_encode` | Produce plain, substituted, transposed, stream-encrypted and encoded versions of the same input, all hand-written | Generate the encrypted and transformed samples the brief asks for, with no crypto library | Modular arithmetic (mod 26 and mod 256), permutation, XOR; substitution keeps the distribution, transposition keeps the counts |
| `profile()` | Computes min-entropy, the order check, chi-square, index of coincidence and printable share in one pass | Gives the evidence that separates cases Shannon entropy cannot | Min-entropy, entropy of differences, chi-square goodness of fit with a Wilson–Hilferty z-score, index of coincidence |
| `random_reference()` | Measures random data of the same length as the sample | Makes small samples comparable instead of judging them against 8 | Expected entropy of a uniform distribution at sample size N |
| `verdict()` | Combines the measures into one reading with reasons | Answers "can statistics tell the samples apart?" | Decision rules over the measured distributions |
| `entropy_map()` | Measures entropy window by window and joins high windows into regions | Locates encrypted or compressed blocks inside a file | Local entropy over a sliding partition of the data |
| `size_experiment()`, `noise_experiment()`, `shape_distribution()` | Vary the sample size, mix in random bytes, or build a k-symbol distribution with a chosen skew | Cover the brief's experiments on size and on changing the distribution | $\log_2 N$ bound; entropy rising as the distribution flattens; $p(i) \propto 1/i^s$ |
| `XorShift` generator | Hand-written 32-bit pseudo-random generator, seeded | Supplies the one-time-pad key, random reference and noise without a library | Uniform pseudo-random sequence used as a reference distribution |
| `main.py` menu | 14 numbered options, gauges, byte map, explanations and a 9-step guided demo | The required menu-driven interface | Presents every result next to its formula and meaning |
| `test_core.py` | 37 known-answer tests | Proves the maths and the ciphers are correct | Checks H(HELLO) = 1.9219, $H = \log_2 n$ for n equally likely symbols, published RC4 and Base64 outputs |

# 6. Results

Substitution and transposition leave entropy unchanged, stream ciphers push it close to the ceiling, and the extra measures separate cases that entropy alone cannot.

**Same input through every method.** Input: the 284-byte demo paragraph, every byte counted.

| Method | Bytes out | Distinct values | H (bits/byte) | ΔH |
|---|---|---|---|---|
| Original data | 284 | 26 | 4.190 | 0.000 |
| Caesar, shift 3 | 284 | 26 | 4.190 | +0.000 |
| Rail fence, 3 rails | 284 | 26 | 4.190 | +0.000 |
| Vigenère, key SECURE | 284 | 30 | 4.546 | +0.356 |
| Base64 | 380 | 52 | 5.326 | +1.136 |
| XOR, repeating key | 284 | 51 | 5.401 | +1.211 |
| Random reference | 284 | 163 | 7.171 | +2.982 |
| One-time pad | 284 | 170 | 7.240 | +3.050 |
| RC4 | 284 | 174 | 7.256 | +3.066 |

With letters A–Z only, the same paragraph gives 4.0852 for the original and 4.5102 for Vigenère, matching the hand calculation in the mathematical model.

**Can the samples be told apart?** The 1,288-byte passage and its transformations, with the verdict the program gives each:

| Sample | H | Order check | Verdict |
|---|---|---|---|
| One symbol repeated | 0.00 | 0.00 | Almost constant |
| English passage | 4.26 | 6.11 | Readable text, or a simple substitution of it |
| Passage, Caesar | 4.26 | 6.12 | Readable text, or a simple substitution of it |
| Passage, Vigenère | 4.64 | 6.43 | Text with flattened letter statistics |
| Passage, Base64 | 5.38 | 6.43 | Encoded text (Base64) |
| Passage, XOR repeating key | 5.62 | 6.96 | Structured binary data |
| Passage, RC4 | 7.85 | 7.82 | Looks random: encrypted, compressed or random data |
| Random reference | 7.85 | 7.87 | Looks random: encrypted, compressed or random data |
| Passage, RC4, sorted | 7.85 | 0.73 | High entropy, but in a predictable order |
| Counter 00 01 02 … | 8.00 | 0.00 | High entropy, but in a predictable order |
| Mixed file, text + encrypted | 7.00 | 7.57 | Mixed: contains a high-entropy block |

Entropy alone separates repetitive, text-like, encoded and random-looking data. It cannot separate plain text from Caesar output, or ciphertext from sorted bytes or a counter. The order check and the index of coincidence do; nothing separates real ciphertext from random data.

**Sample size.** RC4 output reads 4.000 bits at 16 bytes, 7.189 at 256, 7.833 at 1,024 and 7.997 at 65,536 bytes. Random data follows the same curve, so small samples always under-report entropy.

**Changing the distribution.** Replacing 0%, 50% and 100% of the paragraph's bytes with random values raises entropy from 4.190 to 6.255 and 7.292 bits.

**Entropy map.** In a 4,716-byte file of text with an encrypted block in the middle, the map flags bytes 1,280 to 3,840 as random-looking. The block really runs from byte 1,288 to 3,864; the difference is the 256-byte window size.

# 7. Demo screenshots

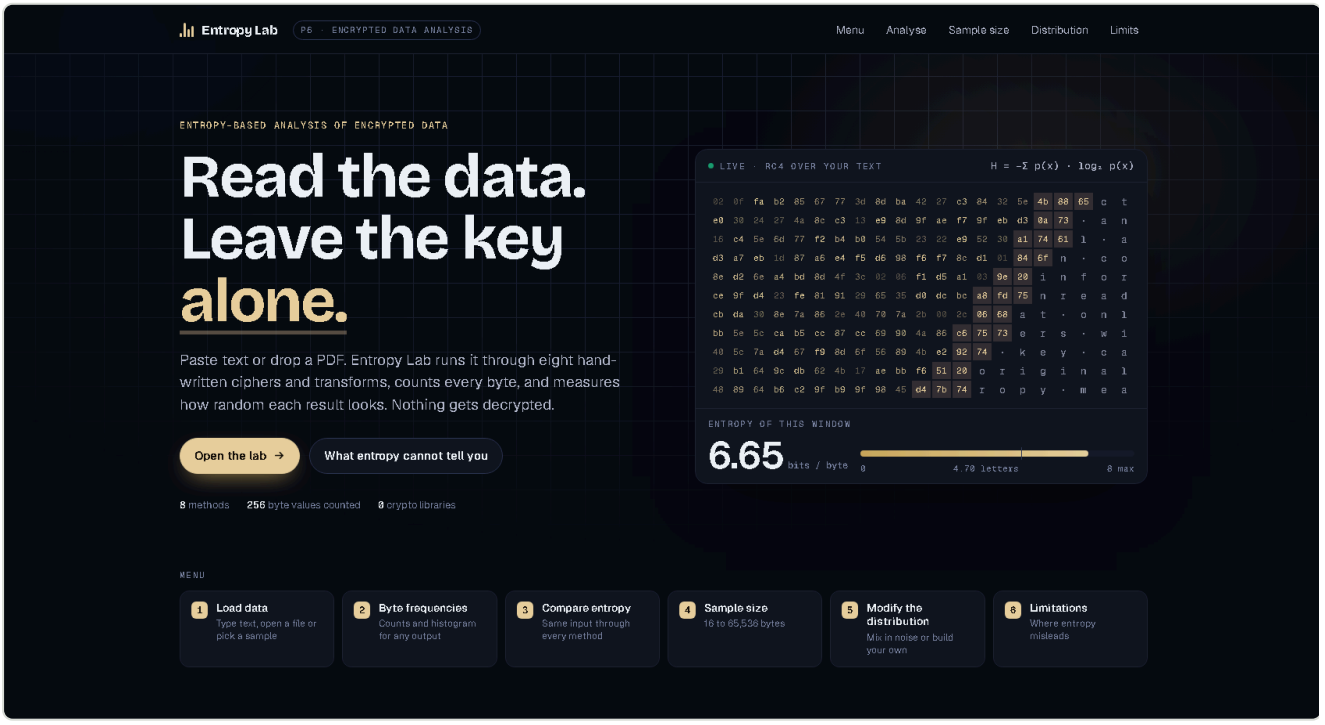



Figure 2: Home screen. Input: the demo paragraph. The live panel encrypts it with RC4 while the entropy of the visible window rises to 6.65 bits per byte. The numbered menu leads to each part of the analysis.

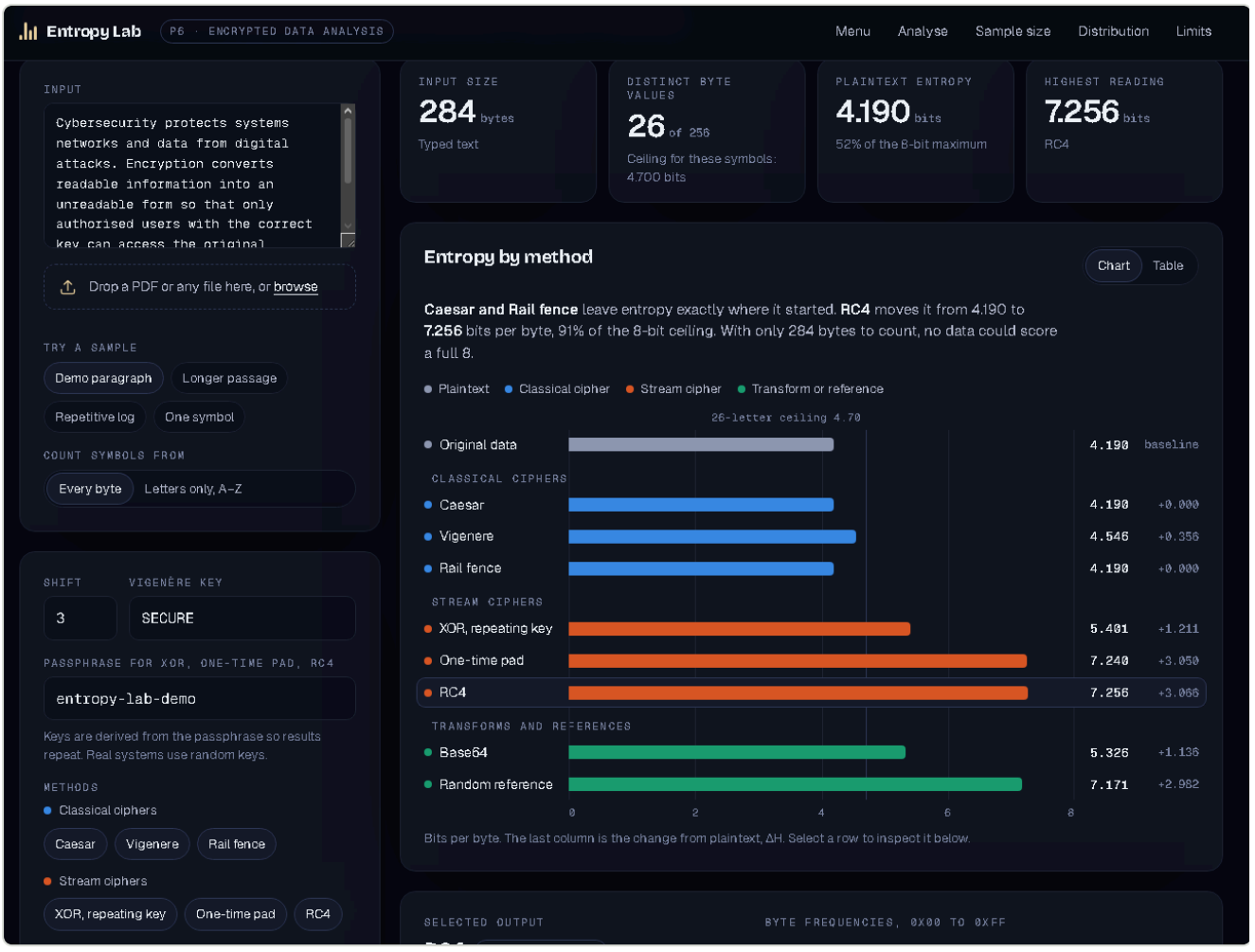



Figure 3: Entropy of the same input after each method. Input: 284 bytes, Caesar shift 3, Vigenère key SECURE. Output: Caesar and Rail fence stay at 4.190 bits; RC4 lifts entropy to 7.256 bits, 91% of the 8-bit ceiling.




Figure 4: Sample-size experiment. At 256 bytes RC4 output reads only 7.189 bits and random bytes 7.111. By 65,536 bytes both reach 8.00, while the text levels off at 4.19.

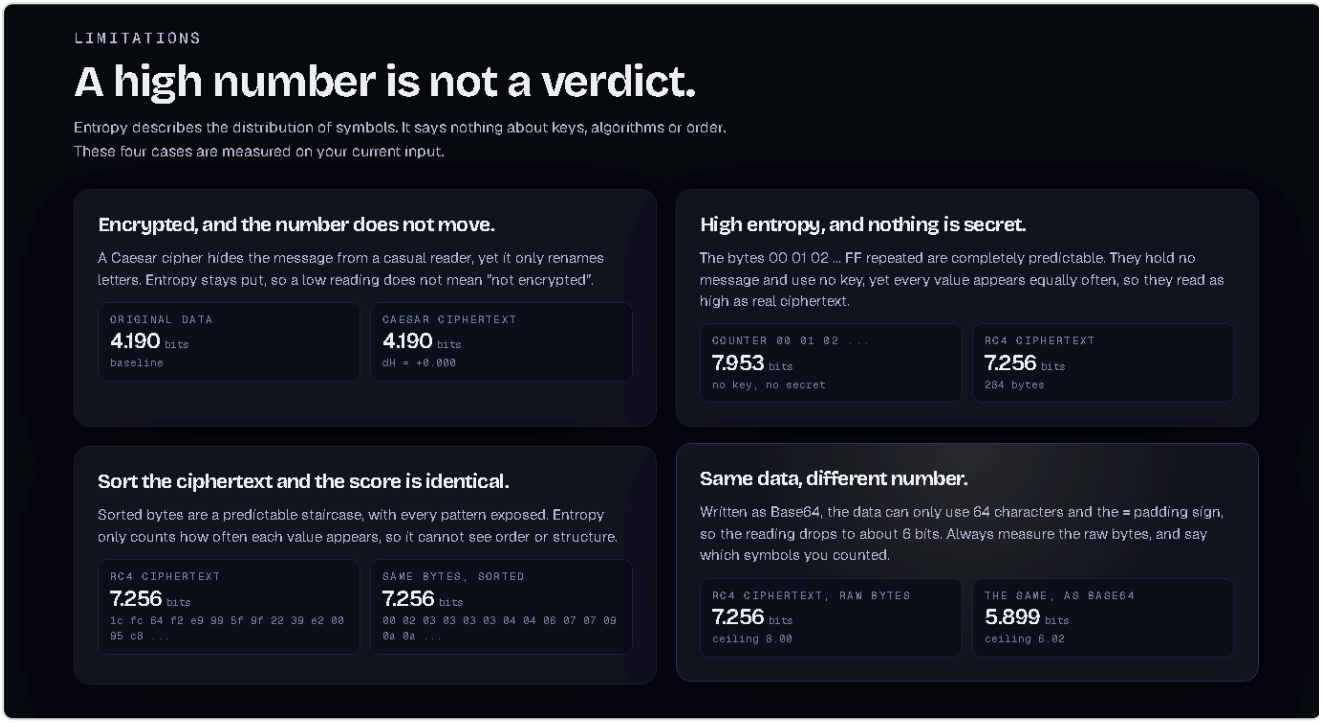



Figure 5: Limitations, measured on the current input. Caesar leaves entropy at 4.190; a predictable counter scores 7.953; sorted ciphertext keeps 7.256; the same ciphertext written as Base64 drops to 5.899.

To be added: a screenshot of the main.py console menu, for example option 4 (profile and verdict), since the menu-driven application is the required interface.

# 8. Individual contribution summary

Lines marked "To be completed" need each member's own details: their secondary role and the evidence of their work.

## Anupama

**Primary role:** Mathematical modelling.

**Secondary role(s):** *To be completed by Anupama.*

**Major contributions:**

- Defined the logical model: plaintext → character frequency → probability → H(X) → encryption → H(Y) → compare.
- Set out the mathematical model: $p(x) = f(x)/N$, $H(X) = -\Sigma\, p(x) \log_2 p(x)$ and $\Delta H = H(Y) - H(X)$.
- Worked the HELLO → KHOOR example by hand (1.922 bits in both), showing that substitution keeps the distribution and why entropy alone cannot prove encryption strength.
- *To be completed by Anupama: any further work.*

**Evidence:** `docs/mathematical_model.pdf` in the repository. *Add commits or other evidence.*

## Hari

**Primary role:** Implementation and coding.

**Secondary role(s):** *To be completed by Hari.*

**Major contributions:**

- Implemented the mathematical model in Python: the hand-written byte-frequency count and Shannon entropy function in `entropy_core.py`, with no library doing the core work.
- Wrote all eight transformations by hand: Caesar, Vigenère, Rail fence, repeating-key XOR, one-time pad, RC4, Base64 and the random reference, plus a seeded xorshift random generator.
- Built the extra measures (min-entropy, order check, chi-square z-score, index of coincidence), the random-data reference, the entropy map and the verdict.
- Built the experiments for sample size, mixing in noise and building a distribution.
- Built the 14-option menu-driven application in `main.py`, with gauges, a byte map, plain-English explanations, a glossary and a 9-step guided demo.
- Built the optional web dashboard (`app.py`, `static/index.html`) used for the demo screenshots.
- Wrote 37 known-answer tests and fixed the issues the test report found: a Windows console encoding crash and the Base64 entropy ceiling.
- Set up the GitHub repository.

**Evidence:** `entropy_core.py`, `main.py`, `test_core.py`, `app.py` and the commits in the repository.

## Karthika

**Primary role:** Algorithm design.

**Secondary role(s):** *To be completed by Karthika.*

**Major contributions:** *To be completed by Karthika: for example, the step-by-step algorithm for computing entropy and comparing methods, and its complexity.*

**Evidence:** *To be completed.*

### Nisha

**Primary role:** Presentation and technical communication.

**Secondary role(s):** *To be completed by Nisha.*

**Major contributions:** *To be completed by Nisha: for example, the presentation deck and how the results are explained to the audience.*

**Evidence:** `docs/G6_Entropy_Analysis_presentation.pdf`. *Add any other evidence.*

### Rohit

**Primary role:** Project coordination and data experimentation.

**Secondary role(s):** *To be completed by Rohit.*

**Major contributions:** *To be completed by Rohit: for example, choosing the data samples and running the sample-size and distribution experiments.*

**Evidence:** *To be completed.*

### Sreerag

**Primary role:** Documentation.

**Secondary role(s):** *To be completed by Sreerag.*

**Major contributions:** *To be completed by Sreerag: for example, this report and the README.*

**Evidence:** *To be completed.*

### Vandana

**Primary role:** Testing and validation.

**Secondary role(s):** *To be completed by Vandana.*

**Major contributions:** *To be completed by Vandana: for example, test cases such as HELLO = 1.922 and Caesar $\Delta H = 0$, and checks of every menu option.*

**Evidence:** *To be completed.*

# 9. GitHub repository and conclusion

**Repository:** github.com/harishankarsa454-code/G6-Entropy-Analysis

| Path | Contents |
|---|---|
| `main.py` | The menu-driven application. Run `python main.py` ; nothing needs installing |
| `entropy_core.py` | Frequency counting, entropy, the extra measures, the transformations and the experiments |
| `test_core.py` | 37 known-answer tests: `python -m unittest test_core -v` |
| `app.py` , `static/` | Optional web dashboard; needs Flask |
| `docs/` | Mathematical model and presentation |
| `README.md` | How to run the application, the team and the menu options |

**Conclusion.** Entropy is a reliable first look at data. It cleanly separates repetitive, text-like, encoded and random-looking samples, and it shows how each transformation changes the distribution. It is not a verdict on security. Caesar and Rail fence hide a message without changing entropy, a counter scores a near-perfect 8 with no secret at all, and small samples read low. Adding the order check, chi-square, the index of coincidence and an entropy map separates most of these cases, but no statistic can tell encrypted data from compressed or random data, or judge how strong the encryption is.